Autonomous Lifecycle & Maintenance Platform

Inverter Anomaly Detection System Design

Document Control

- **Product Name:** Airobotix Omni-Grid Platform
- **Version:** 1.0
- **Date:** April 2026
- **Design By:** Kazi Abdul Mubin (AI and Data Analyst Intern)
- **Audience:** Data Scientists, ML Engineers, AI Research Teams
- **Classification:** Internal - Strategic

1. Introduction and Problem Context

Modern solar plants depend on the stable operation of inverters because inverters convert DC power from solar panels into AC power suitable for the grid. If an inverter begins to behave abnormally, the result can be energy loss, reduced plant efficiency, increased maintenance cost, and, in serious cases, full equipment failure. Because of this, inverter monitoring is not only a maintenance issue but also a business and reliability issue.

The purpose of this design is to create a **real-time inverter anomaly detection system** that identifies abnormal operating behaviour early, classifies the most likely fault type, and supports faster maintenance decisions. According to the uploaded model card, the system is designed as a two-stage architecture using **Isolation Forest** for anomaly detection and **XGBoost** for supervised fault classification. The documented training data spans 47 solar plants across Australia, includes 8.2 million normal operation records and 12,400 labelled anomalies, and covers several inverter manufacturers such as SMA, Fronius, Huawei, SolarEdge, and ABB.

This makes the anomaly detection use case highly valuable from both an engineering and machine learning perspective because it combines real-time telemetry, severe class imbalance, explainability, low-latency inference, and production MLOps. The design goal is therefore not simply to predict unusual behaviour, but to build a complete operational monitoring system that transforms streaming telemetry into maintenance-ready intelligence.

2. Design Objectives

The anomaly detection system is intended to satisfy several key objectives.

First, it must support **early anomaly detection** so that abnormal inverter behaviour is recognised before a complete failure occurs. This reduces downtime and helps prevent energy loss.

Second, it must support **fault classification** after an anomaly has been detected. The uploaded model card defines nine possible fault classes, including DC overvoltage, AC grid fault, overtemperature, insulation failure, communication loss, MPPT malfunction, fan failure, capacitor degradation, and other faults.

Third, the design must support **real-time or near-real-time inference**. The platform architecture in the uploaded file uses Azure Event Hubs for ingestion, stream processing for hot-path feature computation, and Azure ML managed endpoints for model serving. It also specifies a p95 latency target below 100 milliseconds for the API.

Fourth, the design must ensure **interpretability and operator trust**. Maintenance planners and operations staff need to understand why an alert was triggered. For this reason, the documented model output includes SHAP explanations showing the most influential features behind each classified fault.

Finally, the design must include **monitoring and retraining capability** so that the system remains reliable as data distributions change over time. This includes drift checks, retraining triggers, version control, and rollback support.

3. High-Level System Architecture

The proposed solution follows a streaming-first Azure-based architecture in which data flows from inverter telemetry to real-time inference, then to alerts and monitoring systems.

3.1 End-to-End Pipeline Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                   SOLAR PLANT DATA SOURCES                                                   │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ SCADA Sensors │ Inverter Telemetry │ Weather APIs │ Maintenance Logs │
└──────────────────────┬──────────────────────────┬──────────────────────┬──────────────────────────────────────┘
                       │                          │                      │
```

```
        ▼           ▼        ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                              DATA INGESTION LAYER                                            │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│ Azure Event Hubs (streaming) │ Azure Data Factory (batch ingestion) │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

                    │
                    ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                              DATA PROCESSING LAYER                                           │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│ Azure Stream Analytics / Flink  │ Databricks │ Great Expectations  │
│ Real-time aggregation          │ Batch ETL  │ Data validation     │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

                    │
                    ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                              DATA STORAGE LAYER                                              │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│ Azure Blob │ InfluxDB / TimescaleDB │ Feast Feature Store │ Synapse │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

```
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                                   ML LAYER                                           │
├──────────────────────────────────────────────────────────────────────────────────────┤
│ Stage 1: Isolation Forest → anomaly score                                            │
│ Stage 2: XGBoost Classifier → fault type + SHAP explanation                          │
└──────────────────────────────────────────┬───────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                              SERVING AND ACTION LAYER                                │
├──────────────────────────────────────────────────────────────────────────────────────┤
│ Azure ML Endpoints │ Alerts │ Dashboards │ Maintenance workflows                    │
└──────────────────────────────────────────┬───────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                            MONITORING AND MLOPS LAYER                                │
├──────────────────────────────────────────────────────────────────────────────────────┤
│ Evidently AI │ Grafana │ Prometheus │ Drift Detection │ Retraining                 │
```

└──┘

This design is directly aligned with the uploaded architecture, which includes Azure Event Hubs, Azure Data Factory, Databricks, Feast, Azure ML, Evidently AI, Grafana, and batch plus real-time model serving layers.

The architecture is strong because it separates data ingestion, processing, storage, model serving, and monitoring into distinct layers. This improves scalability, maintainability, and reliability. It also supports both streaming inference and batch retraining, which is essential in production ML systems.

4. Model Flow and Decision Logic

The anomaly detection solution uses a **two-stage hybrid model architecture** rather than a single classifier.

4.1 Model Flow Diagram

```
Input Features (42 telemetry + engineered features)
        |
        ▼
   Robust Scaling
        |
        ▼
  Isolation Forest Model
        |
        ▼
  Anomaly Score (0 to 1)
        |
   ┌────────┴────────┐
   |                 |
score < threshold  score ≥ threshold
   |                 |
   ▼                 ▼
Normal        Trigger Stage 2
                     |
                     ▼
        XGBoost Fault Classifier
                     |
                     ▼
       Fault Type Prediction (9 classes)
                     |
                     ▼
```

SHAP Explanation Layer

|

▼

Recommended Action + Severity + Top Drivers

According to the model card, Stage 1 applies **Isolation Forest with 200 estimators and contamination of 0.05**, and uses an anomaly threshold of **0.7** to decide whether the record should be flagged. If the record is flagged as anomalous, Stage 2 then applies an **XGBoost classifier with max depth 8 and 300 estimators** to predict the fault type.

This two-stage approach is a strong design choice. The first reason is that anomalies are extremely rare compared with normal behaviour. The class distribution shows that normal operation represents about 99.85% of the data. A purely supervised model would struggle under such class imbalance.

The second reason is that the unsupervised first stage can detect unusual behaviour even when labels are incomplete. This is practical in industrial systems where not every abnormal pattern is perfectly labelled.

The third reason is that the supervised second stage adds operational value by identifying the likely fault type. This makes the system much more actionable than a simple anomaly alarm.

5. Data Inputs and Feature Engineering

5.1 Raw Inputs

The anomaly detection model uses 42 input dimensions based on real-time telemetry and engineered operational features. The raw telemetry includes DC voltage, DC current, DC power, AC output voltage, AC current, AC power, AC frequency, power factor, inverter efficiency, internal temperature, heatsink temperature, and ambient temperature.

These features are well chosen because inverter faults often appear first as instability in voltage, current, thermal behaviour, or efficiency.

5.2 Engineered Inputs

The uploaded model card also includes several engineered features:

- rolling means over short and medium windows
- rolling standard deviations
- rate-of-change variables

- efficiency deviation from expected values
- temperature stress index
- voltage imbalance ratio
- current imbalance ratio
- frequency deviation from 50 Hz
- cumulative operating hours
- time since last restart
- days since last maintenance
- weather-normalised performance ratio

These engineered features are especially important because anomalies in industrial equipment are often not visible from a single sensor reading. For example, a high temperature by itself may not indicate failure, but rising temperature combined with falling efficiency and increasing variability is much more informative.

5.3 Why Feature Engineering Matters Here

The feature design reflects strong industrial ML reasoning.

First, **temporal context matters**. A single reading cannot show whether a system is drifting, oscillating, or rapidly degrading. Rolling statistics and deltas help capture this.

Second, **domain-aware indicators matter**. Features such as efficiency deviation and thermal stress index are not generic statistical transformations; they reflect engineering knowledge about how inverters degrade.

Third, **operational context matters**. Variables such as cumulative hours, restart frequency, and days since maintenance connect equipment state with lifecycle behaviour.

6. Data Pipeline Design

6.1 Real-Time Pipeline

The uploaded architecture uses **Azure Event Hubs** for real-time ingestion of IoT and SCADA data, including protocols such as MQTT, OPC-UA, and Modbus TCP. It also uses Azure Stream Analytics or Apache Flink for real-time transformation and aggregation.

The streaming path can be described as follows:

Inverter telemetry from SCADA systems is sent through an IoT edge gateway into Azure Event Hubs. Stream processing then aggregates recent telemetry, applies feature

transformations, and prepares the data for the inference endpoint. The processed feature vector is then sent to the deployed anomaly detection model, which returns an anomaly score, fault type if needed, and explanation metadata.

This is a strong design because it supports low latency and continuous scoring while separating ingestion from inference.

6.2 Batch Pipeline

The uploaded platform architecture also defines a batch pipeline using **Azure Data Factory** and **Azure Databricks** for ETL, historical feature generation, retraining, and data validation.

In the batch path, raw data stored in Azure Blob Storage is processed in Databricks, validated using Great Expectations, and transformed into reusable features for training and evaluation. These features are then stored in the feature store and used for retraining pipelines in Azure ML.

This batch path is essential because the streaming system can only score data, while the historical system is responsible for learning from new data, updating features, recalibrating the model, and supporting model evaluation.

6.3 Feature Store Consistency

The architecture uses **Feast** as the feature store. This is important because it helps reduce training-serving skew by ensuring that the same engineered features are available during training and online inference.

This improves consistency, reproducibility, and operational stability.

7. Azure-Based Technical Design

Azure is an appropriate cloud platform for this solution because the uploaded design is already built around Azure-native services:

- Azure Event Hubs for streaming ingestion
- Azure Data Factory for scheduled ingestion and orchestration
- Azure Blob Storage for raw and archived data
- Azure Databricks for scalable feature engineering
- Azure ML for model training and deployment
- Azure ML managed endpoints for inference
- Azure-based monitoring and storage integration

This creates a coherent production architecture in which the major platform components are compatible with each other.

7.1 Deployment Design

The anomaly detection model is documented as being deployed on **Azure ML Managed Endpoints**, with autoscaling between 2 and 10 instances, Docker-based packaging, and multi-region deployment across AU East and AU Southeast. The compute target is listed as Standard_D4s_v3.

This deployment design is effective because it supports:

- high availability
- load-based scaling
- rapid deployment of new model versions
- resilience across regions

7.2 API Design

The uploaded file defines the API input as inverter ID, timestamp, and a features object, and the output as anomaly score, anomaly flag, fault type, confidence, explanation, and model version.

This is a good interface design because it combines prediction, confidence, explainability, and traceability in one response. Including the model version is particularly useful for production debugging and audit trails.

8. SHAP Explainability Design

Explainability is a critical part of this system because maintenance teams need to understand why the model produced an alert. According to the uploaded model card, SHAP explanations are included for anomaly-triggered classifications, with top contributing features listed in the output.

A simple way to represent the SHAP output in the design is shown below.

8.1 SHAP Contribution Diagram

SHAP Contribution Example: DC_OVERVOLTAGE

Feature Contribution

---

dc_voltage +0.42

voltage_rate_of_change_5min +0.28

efficiency_deviation +0.15

temperature_stress_index +0.06

power_factor_deviation -0.02

Total pushes prediction toward: HIGH DC_OVERVOLTAGE risk

This style of explanation is valuable because it translates a raw prediction into an understandable engineering story. Instead of only saying that a fault exists, the system shows which features pushed the decision most strongly.

8.2 Operational Importance of SHAP

SHAP adds value in four ways.

First, it improves **operator trust**, because the user can see why the model produced the result.

Second, it improves **troubleshooting speed**, because the maintenance team can focus on the most relevant signals first.

Third, it improves **model monitoring**, because changes in important features over time may indicate drift or sensor issues.

Fourth, it improves **governance and responsible AI**, because decisions affecting maintenance actions become more transparent.

9. Evaluation Framework and Metric Justification

9.1 Why Accuracy Alone is Misleading

Accuracy is not a suitable primary metric for this use case because the vast majority of records represent normal operation. Since roughly 99.85% of records are normal, a trivial model could achieve high accuracy while missing almost every meaningful anomaly.

For this reason, the evaluation framework must focus on rare-event detection quality, operational usefulness, and business cost.

9.2 Metrics for the Anomaly Detection Stage

The uploaded model card reports the following validation performance for the unsupervised anomaly detection stage:

- Precision: 87.3%
- Recall: 92.1%
- F1 score: 89.6%
- False positive rate: 0.8%

These metrics are appropriate for the following reasons.

Precision is important because too many false alarms can overwhelm the operations team and reduce trust in the system.

Recall is important because missing a real failure is costly and can lead to downtime, energy loss, and potentially larger maintenance events.

F1 score is important because it balances precision and recall.

False positive rate is important because even if recall is high, operational overload must remain low.

The fact that recall is slightly higher than precision makes sense in this context because the design prioritises missed failures as more dangerous than unnecessary inspections.

9.3 Metrics for Fault Classification

For the supervised classifier stage, the uploaded model card reports:

- Macro F1 = 0.891
- Weighted F1 = 0.923
- Top-1 Accuracy = 89.4%
- Top-3 Accuracy = 96.7%

These are well-justified choices.

Macro F1 is useful because it evaluates performance across all classes more evenly, including rare fault classes.

Weighted F1 gives a general summary while still reflecting class support.

Top-3 accuracy is especially valuable in maintenance operations because if the most likely class is not correct, surfacing the top few probable fault types can still guide diagnosis and inspection.

9.4 Temporal Evaluation

The uploaded methodology also uses temporal train, validation, and test splits, expanding-window time-series cross-validation, geographic testing, seasonal evaluation, and degraded sensor scenarios.

This is important because inverter telemetry is time-dependent. Random splitting would cause data leakage and produce unrealistic performance estimates.

10. Monitoring, MLOps, and Retraining

10.1 Monitoring Architecture

The uploaded file recommends monitoring:

- weekly data drift using KS tests
- daily anomaly rate tracking
- monthly precision and recall on labelled production data
- real-time latency tracking
- infrastructure monitoring with Grafana and Prometheus
- model monitoring with Evidently AI

A simplified monitoring view is shown below.

```
Production Predictions
      │
      ▼
 Log features + outputs
      │
      ├──► Data drift checks
      ├──► Prediction distribution monitoring
      ├──► Latency monitoring
      ├──► Error rate monitoring
      └──► Label-based performance review
```

This monitoring layer is essential because even a strong model will degrade if data distributions change, sensor quality declines, or new equipment types enter the fleet.

10.2 Retraining Triggers

The uploaded model card specifies retraining triggers such as:

- precision below 80% for two consecutive weeks
- significant data drift
- introduction of a new inverter type
- quarterly scheduled retraining

These triggers are appropriate because inverter fleets and environmental conditions evolve over time. A static model would eventually become outdated.

10.3 Versioning and Rollback

The system also includes semantic versioning, A/B testing, and rollback capability within five minutes.

This is a strong production design because it allows safer deployment of new versions without exposing the entire fleet to unnecessary risk.

11. Limitations and Risk Considerations

A good HD-level design should not ignore limitations. The uploaded model card identifies several:

- lower performance for very rare fault types
- weaker generalisation to unseen inverter models
- performance degradation during extreme weather events
- sensitivity to communication delays
- need for recalibration under sensor drift

These are realistic and important.

In addition, several practical risks should be considered.

One risk is **alert fatigue**, where too many alerts reduce trust and responsiveness.

Another is **manufacturer bias**, because some inverter types are more represented in the training data than others.

Another is **data quality risk**, since the model depends on continuous and reliable telemetry.

Finally, there is the risk of **over-reliance on model output**, which is why human oversight remains necessary for critical decisions.

12. Recommendations for Future Improvement

Several future improvements could strengthen this system further.

One improvement would be **active learning**, where novel or uncertain cases are sent to experts for labelling and then used to improve the supervised classifier.

Another would be **synthetic augmentation for rare faults**, which may help improve macro-level performance on underrepresented classes.

A third would be **manufacturer-specific fine-tuning**, so the system can adapt more effectively to new inverter models.

A fourth would be **dynamic thresholding**, replacing the fixed anomaly threshold with thresholds that vary by site, season, or operating context.

A fifth would be **edge inference**, where lightweight anomaly scoring is pushed closer to the device layer for faster local action in remote sites.

13. Conclusion

The proposed inverter anomaly detection system is a strong example of a production-ready ML design because it combines a clear business objective, a robust two-stage modelling strategy, explainability, scalable Azure deployment, and strong monitoring practices.

The first stage uses unsupervised anomaly detection to identify abnormal patterns in highly imbalanced telemetry data. The second stage adds actionable fault classification and SHAP-based transparency. Around these models, the Azure-based data platform supports both real-time inference and historical retraining, while the monitoring layer ensures the model remains reliable over time.

Overall, this design goes beyond a simple model description and presents a full operational ML system for industrial anomaly detection in solar infrastructure.